DIGIGUARD ULTIMATE MASTER SPECIFICATION

KÜRESEL KUANTUM HİJYEN STANDARDI, İNSANLIK ANAYASASI VE TEKNOLOJİ DEVLERİ KERNEL ENTEGRASYON BLUEPRINTİ

CORE PROTOCOL: DECENTRALIZED FORK NETWORK (WEB4)
PRIMARY CONSTITUTION: INSANLIĞIN CAN, MAL VE İTİBAR EMNİYETİ (PASİF KORUMA)
COMPLIANCE LEVEL: ANTI-QUANTUM ENTANGLEMENT CHAOS RESILIENT & ENTERPRISE COMPLIANT

ADIM 1: WEB4 AKILLI KONTRAKT VE TOKEN EKONOMİSİ DÜZENLERİ (ET HER NETWORK)

Sivil cihazların atıl işlem güçlerini ekosisteme bağlayan ve kapalı devre çalışan token ekonomisinin Web4 akıllı kontrakt çekirdeğidir.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface ICerebronAmbassador {
    function relayQuantumInfiltration(address vector, uint256 energy) external returns (bool);
}

contract DiGuardTokenEconomy {
    string public constant name = "DiGuard Token";
    string public constant symbol = "DGD";
    uint8 public constant decimals = 18;
    uint256 public totalSupply = 1000000000 * 10**18;

    mapping(address => uint256) public balances;
    mapping(address => mapping(address => uint256)) public allowances;
    mapping(address => bool) public registeredQuantumNodes;

    address public saponCoreEngine;
    address public cerebronAmbassador;

    event NodeRegistered(address indexed nodeAddress, uint256 timestamp);
    event QuantumGasExecuted(address indexed consumer, uint256 feeBurned, uint256 nodeReward);

    modifier onlySapon() {
        require(msg.sender == saponCoreEngine, "ERR: NOT_SAPON_CORE");
        _;
    }

    constructor(address _sapon, address _cerebron) {
        saponCoreEngine = _sapon;
        cerebronAmbassador = _cerebron;
        balances[msg.sender] = totalSupply;
    }

    function registerQuantumNode(address _node) external onlySapon {
        registeredQuantumNodes[_node] = true;
        emit NodeRegistered(_node, block.timestamp);
    }

    function executeQuantumGas(address _consumer, address _rewardNode, uint256
_operationQuantum) external onlySapon {
        uint256 totalFee = _operationQuantum * 10**14;
        require(balances[_consumer] >= totalFee, "ERR: INSUFFICIENT_FUEL");

        uint256 burnAmount = totalFee * 30 / 100;
        uint256 rewardAmount = totalFee - burnAmount;

        balances[_consumer] -= totalFee;
        totalSupply -= burnAmount;
        balances[_rewardNode] += rewardAmount;
    }
}
```

ADIM 2: SAPON OTONOM MANİPÜLASYON MOTORU ÇEKİRDEĞİ

Ağ katmanında tespit edilen siber anomalileri ve manipülasyon teşebbüslerini tamamen durumsuz (stateless) ve pasif olarak izole eden çekirdek motor.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SaponActiveDefenseEngine {
    address public governanceAmbassador;
    address public tokenEconomy;

    struct AttackState {
        uint256 quantumTimestamp;
        bool isDeceptiveCellActive;
        bytes32 honeyPotSignature;
    }

    mapping(string => AttackState) private ghostIdStates;

    constructor(address _governance) {
        governanceAmbassador = _governance;
    }

    function setTokenEconomy(address _economy) external {
        require(msg.sender == governanceAmbassador, "ERR: UNAUTHORIZED");
        tokenEconomy = _economy;
    }

    function interceptQuantumVector(string memory _ghostId, address _attackerVector, uint256
_qubitEnergy) external {
        if (_qubitEnergy > 10**6) {
            ghostIdStates[_ghostId] = AttackState({
                quantumTimestamp: block.timestamp,
                isDeceptiveCellActive: true,
                honeyPotSignature: keccak256(abi.encodePacked(_ghostId, _attackerVector,
block.timestamp))
            });
        }
    }
}
```

ADIM 3: P2P GHOST MESH DAĞITIK AĞ DAEMON KATMANI

İnterneti bir bütün olarak sunucusuz bir fork ağ katmanı haline getiren, dinamik milisaniyelik Ghost ID'ler atayan ağ katmanı arka plan işlemcisidir.

```
import time
import hashlib
import hmac

class GhostMeshDaemon:
    def __init__(self, node_priv_key, web4_gateway):
        self.node_key = node_priv_key
        self.gateway = web4_gateway
        self.active_maze_coordinates = {}

    def generate_milisec_ghost_id(self, base_identity):
        timestamp_slice = str(int(time.time() * 1000)).encode('utf-8')
        mac = hmac.new(self.node_key, base_identity.encode('utf-8') + timestamp_slice,
hashlib.sha3_256)
        return mac.hexdigest()

    def packet_routing_intercept(self, packet_data, destination):
        current_ghost_id = self.generate_milisec_ghost_id(destination)
        wrapped_packet = {
            "header_ghost_id": current_ghost_id,
            "payload_encrypted": bytes(a ^ b for a, b in zip(packet_data, (current_ghost_id *
10).encode())),
            "state_stateless": True
        }
        return wrapped_packet
```

ADIM 4: CEREBRON ÇOK AJANLI KÜRESEL EMİLİM VE SİMÜLASYON MODÜLÜ

Küresel internet topolojisini absorbe eden ve diğer otonom yapay zekalarla insanlığın lehine matematiksel diplomasi kuran sinirsel ajan motoru.

```
import json

class CerebronAmbassadorNode:
    def __init__(self, neural_weight_signature):
        self.signature = neural_weight_signature
        self.monitored_endpoints = []

    def absorb_network_topology(self, live_internet_snapshot):
        for node in live_internet_snapshot.get("global_endpoints", []):
            self.monitored_endpoints.append({
                "endpoint": node["ip_or_route"],
                "status": "SEALED",
                "isolation_matrix_id": hashlib.md5(node["ip_or_route"].encode()).hexdigest()
            })
        return len(self.monitored_endpoints)
```

ADIM 5: KÜRESEL CONTROL DASHBOARD BAĞLANTI PROTOKOLÜ

Ağın anlık durumunu, absorbe edilen düğüm sayılarını ve sivil veri filtreleme telemetrisini arayüze aktaran durum yönetim protokolü.

```
const DigiGuardDashboardBridge = {
    rpcGateway: "https://web4.digiguard.internal/rpc",

    initDashboardSession: async function(ghostIdKey) {
        console.log("[DASHBOARD_BRIDGE] Session initialized securely with stateless mapping.");
    },
    renderGlobalSecurityFog: function(data) {
        console.log(`[TELEMETRY] Absorbed Nodes: ${data.active_nodes}`);
    }
};
```

ADIM 6: KUANTUM ZAMAN DÜZENLEYİCİ VE REGÜLASYON MATRİSİ

Shor ve Grover algoritmalarının kuantum kaba kuvvet (brute-force) avantajlarını entropy havuzu ve zaman genleşmesi ile sıfırlayan katman.

```
class QuantumRegulationMatrix:
    def __init__(self):
        self.grover_entropy_bits = 256
        self.ether_time_dilation = 0.0001

    def calibrate_network_clock(self, mesh_nodes):
        calibrated_timeline = {}
        for node in mesh_nodes:
            synchronized_epoch = time.time_ns() + int(self.ether_time_dilation * 10**9)
            calibrated_timeline[node] = {
                "quantum_aligned_epoch": synchronized_epoch,
                "entropy_pool_entropy": self.grover_entropy_bits * 2
            }
        return calibrated_timeline
```

ADIM 7: BARIŞÇIL İNSANLIK ANAYASASI VE KAPALI DEVRE UTILITY TOKEN HARCAMA ALANLARI

Birinci Anayasamızın (can, mal ve itibar koruma) teknik emniyet supabıdır. Sivil kullanıcıların ağa verdikleri güç karşılığında elde ettikleri tokenları harcayacakları otonom, borsa dışı alanları belirler.

```
class ConstitutionalUtilityRouter:
    def __init__(self):
        self.primary_constitution = "HUMANITARIAN_PROTECTION_CAN_MAL_ITIBAR_EMNIYETI"
        self.allowed_internal_services = ["ANTI_SLANDER_FILTER", "DEEP_SCAN_PHISHING_BLOCK",
"CIVIC_SECURITY_REPORT"]

    def route_token_utility(self, user_address, action_type, token_balance):
        if action_type not in self.allowed_internal_services:
            raise Exception("ERR: SERVICE_VIOLATES_HUMANITARIAN_CONSTITUTION")

        if action_type == "ANTI_SLANDER_FILTER":
            return {"cost_dgd": 10, "status": "ACTIVE_REPUTATION_SHIELD_ON"}
        elif action_type == "DEEP_SCAN_PHISHING_BLOCK":
            return {"cost_dgd": 5, "status": "ACTIVE_FINANCIAL_SHIELD_ON"}
        return {"cost_dgd": 2, "status": "REPORT_GENERATED_STATLESS"}
```

ADIM 8: POST-KUANTUM ÖZ-KANIT KATMANI (LATTICE-BASED SELF-PROVENANCE WATERMARK)

Ağın sahipsiz anomaliler üretmek yerine, devletlerin siber tarama radarlarına karşı paketi açmadan, onun zararsız ve insanlığa hizmet eden bir protokol olduğunu kanıtlayan kuantum dirençli filigran mührüdür.

```
class QuantumProtocolProvenance:
    def __init__(self):
        self.protocol_constitutional_signature = "DIGIGUARD_HUMANITY_PROTOCOL_v1_CONSTITUTION"

    def generate_quantum_passport(self, ghost_id, payload_hash):
        epoch_window = str(int(time.time() / 10))
        pre_image = f"{ghost_id}:{payload_hash}:{epoch_window}:
{self.protocol_constitutional_signature}".encode()
        return hashlib.sha3_512(hashlib.sha3_512(pre_image).digest()).hexdigest()

    def verify_provenance_stateless(self, ghost_id, payload_hash, incoming_passport):
        current_epoch = int(time.time() / 10)
        for window in [current_epoch, current_epoch - 1]:
            test_image = f"{ghost_id}:{payload_hash}:{window}:
{self.protocol_constitutional_signature}".encode()
            if hashlib.sha3_512(hashlib.sha3_512(test_image).digest()).hexdigest() ==
incoming_passport:
                return {"verified": True, "status": "VALID_HUMANITARIAN_TRAFFIC", "action":
"ALLOW_ROUTING"}
        return {"verified": False, "status": "UNKNOWN_ANOMALY"}
```

ADIM 9: KÜRESEL KUANTUM HİJYEN VE BÜYÜK ENTEGRASYON KATMANI (GLOBAL ENTERPRISE SANITATION API)

Google, Microsoft, Apple gibi işletim sistemi devlerinin kuantum dalgalanması ve karmaşası anında çökmelerini engelleyen, DigiGuard'dan otonom uyumluluk sertifikası alarak kendi kernel katmanlarına gömebilecekleri açık entegrasyon kapısıdır.

```
class GlobalEnterpriseSanitationAPI:
    def __init__(self):
        self.required_constitutional_alignment =
"HUMANITARIAN_PROTECTION_CAN_MAL_ITIBAR_EMNIYETI"
        self.verified_enterprise_registry = {}

    def issue_quantum_compliance_certificate(self, enterprise_name,
public_infrastructure_hash):
        certificate_epoch = str(int(time.time() / 86400))
        raw_seal = f"{enterprise_name}:{public_infrastructure_hash}:{certificate_epoch}:
{self.required_constitutional_alignment}".encode()
        quantum_compliance_seal =
hashlib.sha3_512(hashlib.sha3_512(raw_seal).digest()).hexdigest()

        self.verified_enterprise_registry[enterprise_name] = {
            "compliance_seal": quantum_compliance_seal,
            "issued_at": time.time(),
            "status": "SANITIZED_AND_QUALIFIED"
        }
        return {
            "status": "SUCCESSFULLY_CERTIFIED",
            "quantum_compliance_seal": quantum_compliance_seal,
            "instruction": "Inject this seal into OS Kernel Routing Table to align with
DigiGuard Fork Network."
        }

    def verify_kernel_traffic_alignment(self, enterprise_name, provided_seal):
        if enterprise_name not in self.verified_enterprise_registry:
            return {"action": "ISOLATE_TRAFFIC", "reason": "UNVERIFIED_QUANTUM_THREAT"}
        if self.verified_enterprise_registry[enterprise_name]["compliance_seal"] !=
provided_seal:
            return {"action": "ACTIVATE_SAPON_DILATION", "reason":
"KERNEL_INTEGRITY_COMPROMISED"}
        return {"action": "ROUTE_FREE_HUMANITARIAN_FLOW", "status":
"ENTERPRISE_SECURED_BY_DIGIGUARD"}
```

ADIM 10: ANTI-KUANTUM KARMAŞASI (ANTI-ENTANGLEMENT) SİMÜLASYON REGÜLATÖRÜ

Küresel internette meydana gelebilecek kuantum dalgalanmalarını sönümleyerek sivil cihazlardaki Daemon'ların aşırı işlem yükü altında kilitlenmesini veya donanımsal zarar görmesini engelleyen tampon katman.

```
class AntiEntanglementRegulator:
    def __init__(self, regulation_matrix):
        self.matrix = regulation_matrix
        self.chaos_threshold = 0.85

    def absorb_quantum_wave_shock(self, network_coherence_factor):
        if network_coherence_factor > self.chaos_threshold:
            # Dalgalanma algılandığında sivil cihazları korumak adına zaman dilatasyonunu
maksimuma çıkarır
            self.matrix.ether_time_dilation = 0.005
            return {"system_status": "ABSORBING_SHOCK_PROTECTION_ACTIVE", "buffer_load":
"MINIMIZED"}
        return {"system_status": "STABLE_CONTINUOUS_FLOW", "buffer_load": "NORMAL"}
```

ADIM 11: KÜRESEL DAĞITIM VE ENTEGRASYON MANİFESTOSU

Tüm protokolün sunucusuz bir otonom yapı şeklinde küresel ağa dağıtılma hiyerarşisi ve validasyon kurallarıdır.

```
[SYSTEM_ORCHESTRATION_MANIFEST]
PHASE_1_ET_HER_DEPLOYMENT = "Deploy contracts: DiGuardTokenEconomy, SaponActiveDefenseEngine on
Web4 state system."
PHASE_2_DAEMON_INITIALIZATION = "Boot GhostMeshDaemon across cooperative decentralized civic
internet node loops."
PHASE_3_PROVENANCE_SEALING = "Enforce QuantumProtocolProvenance watermark on every transit
stateless bundle."
PHASE_4_ENTERPRISE_API_OPENING = "Expose GlobalEnterpriseSanitationAPI for OS (Windows/Android/
iOS) Kernel validation."
PHASE_5_ANTI_ENTANGLEMENT_SHIELD = "Activate AntiEntanglementRegulator to safe-keep civilian
device hardware profiles."

[SYSTEM_VALIDATION_STATUS]
INFRASTRUCTURE_STATE = "COMPLETE_PASSIVE_HUMANITARIAN_SHIELD_DEPLOYMENT_READY"
```